

Docker Compose: Introducción e uso básico

1. Que é Docker Compose

Docker Compose é unha ferramenta que permite **definir e executar aplicacións multi-contedor** mediante un único ficheiro de configuración YAML (`docker-compose.yml`).

En lugar de executar varios comandos `docker run` manualmente, con Compose podes **declarar servizos, redes e volumes** nun só ficheiro e levantar todo o entorno con:

```
docker compose up
```

2. Para que serve

- **Organizar contedores relacionados** (por exemplo, unha app web + base de datos).
- **Automatizar despregues locais** e reproducibles.
- **Definir configuracións complexas** (portos, variables de entorno, volumes, etc.) nun só documento.
- **Evitar erros humanos** e facilitar colaboración entre desenvolvedores.

3. Sintaxe básica do ficheiro `docker-compose.yml`

Estrutura xeral:

```
version: "3.9"           # Versión da sintaxe de Docker Compose
services:                 # Lista de servizos (cada un é un contedor)
  nome_servizo:
    image: imaxe:etiqueta
    container_name: nome_contedor
    ports:
      - "host:contedor"
    environment:
      - VARIABLE1=valor1
    volumes:
      - ./ruta_local:/ruta_contedor
    depends_on:
      - outro_servizo
volumes:                  # (opcional) Definición de volumes persistentes
networks:                 # (opcional) Definición de redes personalizadas
```

Elementos principais:

Clave	Explicación
<code>version</code>	Define a versión da especificación de Docker Compose.

Clave	Explicación
<code>services</code>	Cada servizo representa un contedor.
<code>image</code>	Indica a imaxe que se vai usar.
<code>build</code>	Permite construír unha imaxe a partir dun Dockerfile local.
<code>ports</code>	Mapea portos do host co contedor.
<code>environment</code>	Engade variables de entorno ao contedor.
<code>volumes</code>	Define volumes compartidos (para persistencia ou código).
<code>depends_on</code>	Indica dependencias entre servizos (ordena o arranque).

4. Exemplo 1: Servidor web simple con Nginx

Estrutura de directorios

```
/compose-web
├── docker-compose.yml
└── index.html
```

index.html

```
<!DOCTYPE html>
<html>
<head><title>Docker Compose</title></head>
<body><h1>Benvido ao servidor Nginx con Docker Compose!</h1></body>
</html>
```

docker-compose.yml

```
version: "3.9"
services:
  web:
    image: nginx:alpine
    container_name: web_nginx
    ports:
      - "8080:80"
    volumes:
      - ./index.html:/usr/share/nginx/html/index.html
```

Execución

```
docker compose up -d
```

Abre o navegador e visita:

🔗 <http://localhost:8080>

Para deter e eliminar os contedores:

```
docker compose down
```

5. Exemplo 2: Aplicación web con Python e base de datos MySQL

Este exemplo amosa como **varios servizos poden traballar xuntos**.

Estrutura do proxecto

```
/compose-flask-mysql
├── app
│   ├── app.py
│   ├── requirements.txt
│   └── Dockerfile
└── docker-compose.yml
```

app/app.py

```
from flask import Flask
import os, mysql.connector

app = Flask(__name__)

@app.route('/')
def index():
    db = mysql.connector.connect(
        host="db",
        user="root",
        password=os.environ.get("MYSQL_ROOT_PASSWORD"),
        database="demo"
    )
    cursor = db.cursor()
    cursor.execute("SELECT 'Docker Compose funcionando!'")
    return cursor.fetchone()[0]

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

app/requirements.txt

```
Flask==3.0.0
mysql-connector-python==9.1.0
```

app/Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

docker-compose.yml

```
version: "3.9"
services:
  web:
    build: ./app
    container_name: flask_app
    ports:
      - "5000:5000"
    environment:
      - MYSQL_ROOT_PASSWORD=1234
    depends_on:
      - db

  db:
    image: mysql:8.0
    container_name: mysql_db
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: 1234
      MYSQL_DATABASE: demo
    volumes:
      - db_data:/var/lib/mysql

volumes:
  db_data:
```

Execución

```
docker compose up -d
```

Logo, accede en:

🔗 <http://localhost:5000>

Verás a mensaxe:

Docker Compose funcionando!

Para deter e limpar todo:

```
docker compose down -v
```

6. Resumo de comandos principais

Comando	Función
<code>docker compose up</code>	Inicia todos os servizos definidos.
<code>docker compose up -d</code>	Inicia en segundo plano (detached).
<code>docker compose down</code>	Detén e elimina contedores e redes.
<code>docker compose ps</code>	Lista os servizos activos.
<code>docker compose logs -f</code>	Mostra os logs en tempo real.
<code>docker compose build</code>	Constrúe imaxes definidas con <code>build</code> .
<code>docker compose exec <servizo> bash</code>	Abre unha shell dentro dun servizo.

7. Boas prácticas

- Empregar versións **estables e concretas** das imaxes (`nginx:alpine`, `mysql:8.0`, etc.).
- Non gardar contrasinais no ficheiro, usar `.env` se é necesario.
- Definir volumes para **persistir datos** de bases de datos.
- Engadir `depends_on` para ordenar a execución dos servizos.
- Empregar `docker compose down -v` para limpar volumes e redes cando fagas probas.